

TD6 : Vues, tables système et rappels SQL — Corrigé enseignant 18968e2

Rosine Cicchetti - Lotfi Lakhal - Mickaël Martin Nevot



Cette œuvre de Rosine Cicchetti est mise à disposition sous licence Creative Commons Attribution - Utilisation non commerciale - Partage dans les mêmes conditions.

Document en ligne : www.mickael-martin-nevot.com

1 Rappel : Prise en main d'un éditeur Oracle

Utilisez une des deux solutions ci-dessous.

1.1 Oracle Live SQL

Vous pouvez utiliser directement, sans installation ou configuration, l'interpréteur en ligne : <https://livesql.oracle.com>.

1.2 Oracle Cloud Free Tier

Mettez en place une solution d'hébergement en ligne d'un SGBD Oracle. Vous pouvez pour cela consulter le document *Vade-Mecum mise en place d'un hébergement Oracle Cloud Free Tier*.

Ajouter ensuite une base de données nommée `zenetude-bd`.

2 Rappel : Généralités

Voici la convention de nommage proposé dans cet enseignement :

- **mots clefs** : en lettre capitales (*upper case*) ;
- **relation** : première lettre de chaque mot en capitale (*pascal case*) ;
- **attributs** : premier mot en minuscule et première lettre de chaque mot suivant en capitale (*camel case*) ;
- **domaine** : en lettre capitales (*upper case*) au format D_XXX¹.

Pour rappel, les valeurs saisies dans une base de données, comme les chaînes de caractères, sont sensibles à la casse et, par convention, sont saisies **en lettres capitales** (*upper case*), sans diacritique, dans le cadre de cet enseignement.

1. Les domaines identiques sont surlignés avec la même couleur.

3 Rappel : Base de données exemple

La base de données exemple, Gestion pédagogique, utilisée dans les documents de travaux dirigés de cet enseignement, permet de gérer une formation en informatique, avec ses professeurs, ses étudiants, ses modules ainsi que leurs enseignements et notations.

3.1 Dictionnaire de données

La base de données Gestion pédagogique a été élaborée à partir du dictionnaire des données suivant² :

Table 1 – Dictionnaire des données de la base de données exemple

Attribut	Description	Domaine	Remarques
numEt	Numéro d'un étudiant	D_NUMET : NUMBER(6,0)	Valeurs uniques
nomEt	Nom d'un étudiant	D_NOM : VARCHAR2(30)	
prenomEt	Prénom d'un étudiant	D_PRENOM : VARCHAR2(20)	
cpEt	Code postal d'un étudiant	D_CP : VARCHAR2(5)	
villeEt	Ville d'un étudiant	D_VILLE : VARCHAR2(20)	
anneeEt	Année d'étude d'un étudiant	D_ANNEE : NUMBER(2,0)	[1..2]
groupeEt	Numéro de groupe d'un étudiant	D_GROUPE : NUMBER(1,0)	[1..5]
numProf	Numéro d'un professeur	D_NUMPROF : NUMBER(3,0)	Valeurs uniques
nomProf	Nom d'un professeur	D_NOM : VARCHAR2(20)	
prenomProf	Prénom d'un professeur	D_PRENOM : VARCHAR2(20)	
adresseProf	Adresse d'un professeur	D_ADRESSE : VARCHAR2(40)	
cpProf	Code postal d'un professeur	D_CP : VARCHAR2(5)	
villeProf	Ville d'un professeur	D_VILLE : VARCHAR2(20)	
specProf	Code d'une matière dont un professeur est spécialiste	D_CODE : VARCHAR2(7)	
code	Code d'un module	D_CODE : VARCHAR2(7)	Valeurs uniques
libelle	Libellé d'un module	D_LIBELLE : VARCHAR2(30)	
heureCMPrev	Nombre d'heures de CM prévues pour un module	D_NBHEURE : NUMBER(3,0)	

2. Les types syntaxiques, utilisés pour la description des domaines, sont disponibles dans Oracle.

Attribut	Description	Domaine	Remarques
heureCMReal	Nombre d'heures de CM réalisées pour un module	D_NBHEURE : NUMBER(3,0)	
heureTPPrev	Nombre d'heures de TP prévues pour un module	D_NBHEURE : NUMBER(3,0)	
heureTPReal	Nombre d'heures de TP réalisées pour un module	D_NBHEURE : NUMBER(3,0)	
discipline	Discipline enseignée	D_DISCIPLINE : VARCHAR2(15)	{INFORMATIQUE, MATHS, GESTION}
coefCC	Coefficient du contrôle continu pour un module	D_COEF : NUMBER(3,0)	[0..100]
coefTest	Coefficient du test pour un module	D_COEF : NUMBER(3,0)	[0..100]
resp	Numéro d'un professeur responsable d'une matière	D_NUMPROF : NUMBER(3,0)	
codePere	Code du module père d'un module	D_CODE : VARCHAR2(7)	
moyCC	Moyenne de contrôle continu d'un étudiant dans un module	D_NOTE : NUMBER(2,0)	[0..20]
moyTest	Moyenne de test d'un étudiant dans un module	D_NOTE : NUMBER(2,0)	[0..20]

Remarques

Les modules forment une hiérarchie entre eux. Quel que soit leur niveau dans la hiérarchie, tous les modules possèdent les mêmes attributs. La racine de l'arbre ainsi formé par cette hiérarchie est le programme pédagogique national de formation en informatique. Il se décompose en véritables modules, eux-mêmes se décomposant en sous-modules et ainsi de suite jusqu'aux feuilles qui correspondent aux matières. Les matières sont les seuls modules pouvant être enseignés et susceptibles de recevoir des notes. Certains modules sont associés à une discipline.

La hiérarchie des modules de l'extension de la base de données exemple est représentée sous forme d'arborescence des codes en figure 1.

Les coefficients de contrôle continu et de test d'un module sont des pourcentages. Ils peuvent ne pas être renseignés. Si un des deux coefficients n'est pas renseigné, l'autre ne doit pas l'être non plus. Leur somme pour un même module est donc soit de 0, soit de 100.

Nous considérons qu'il n'y a pas deux personnes homonymes ayant le même prénom et le même nom.

3.2 MCD

Le MCD (voir figure 2) modélise trois entités : **Module** (les unités d'enseignement organisées en hiérarchie), **Étudiant** et **Prof**. L'association **Notation** relie un étudiant à un module et porte ses moyennes de contrôle continu et de test. L'association ternaire **Enseignement** lie un

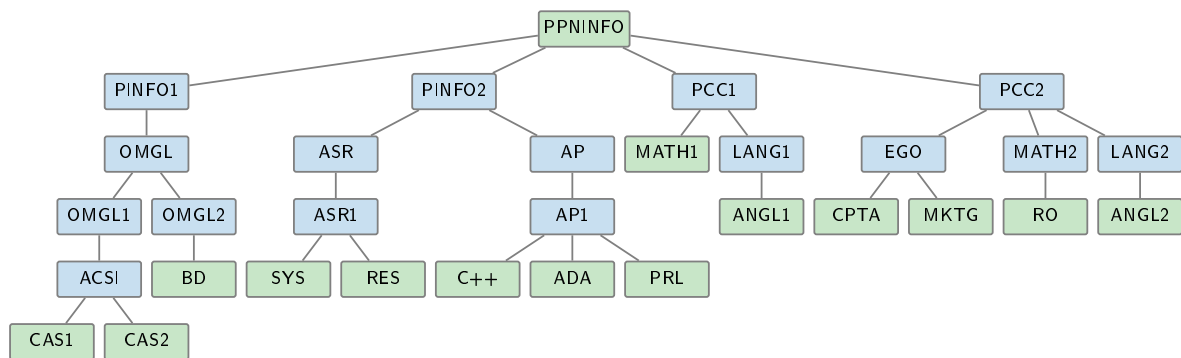


Figure 1 – Hiérarchie des modules

professeur, un module et un étudiant. Un professeur peut être **Spécialiste** d'un module (sa discipline de référence) et **Responsable** d'un module. Enfin, l'association réflexive **A pour père** matérialise la hiérarchie entre modules (voir figure 1).

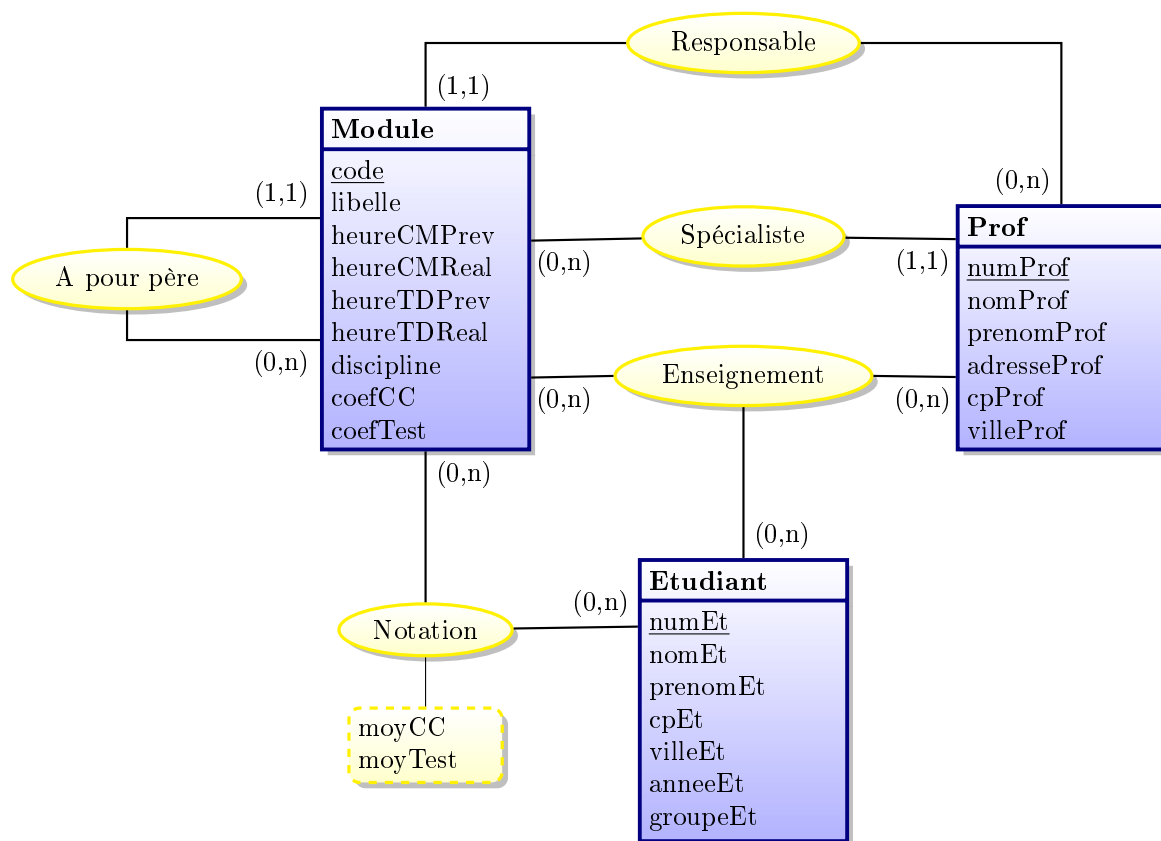


Figure 2 – Modèle conceptuel des données (MCD)

3.3 Schéma relationnel

Les clefs primaires sont soulignées et les clefs étrangères en *italique suivies d'un #*.

Le schéma relationnel de la base de données exemple est présenté ci-après :

Etudiant (numEt, nomEt, prenomEt, cpEt, villeEt, anneeEt, groupeEt)

Professeur (numProf, nomProf, prenomProf, adresseProf, cpProf, villeProf, *specProf#*)

Module (code, libelle, heureCMPprev, heureCMReal, heureTDPrev, heureTDReal, discipline, co-

efCC, coefTest, resp#, codePere#)

Note (numEt#, code#, moyCC, moyTest)

Enseigne (numEt#, code#, numProf#)

Remarques

La clef étrangère **resp** dans **Module** représente le numéro d'un professeur responsable d'un module^a et fait référence à la clef primaire **numProf**.

La clef étrangère **specProf** dans **Professeur** représente le code d'une matière dont un professeur est spécialiste et fait référence à la clef primaire **code**.

La clef étrangère **codePere** dans la relation **Module** fait référence à la clef primaire **code**.

^a. Idéalement, il ne devrait être possible d'y avoir qu'un responsable d'une matière, pas de n'importe quel module. Par souci de simplification, cette contrainte n'a pas été prise en compte dans l'intension de la base de données exemple.

Les autres clefs étrangères font référence aux clefs primaires de même nom.

4 Requêtes avec SQL

Formulez, en SQL, sur la base de données exemple, les requêtes d'interrogation suivantes.

4.1 Création et manipulation de vues

Répondez aux questions suivantes sous forme de requêtes de création de vues, avant de consulter chaque vue par une requête d'interrogation ou des tentatives d'insertion, une valide et une invalide, pour les vues de mise à jour. Les vues à créer ont pour objectifs de créer de nouvelles relations logiques répondants à des besoins fréquents, d'assurer la prise en compte des contraintes d'intégrité de la base ou de limiter la consultation des données pour certains utilisateurs.

Q1 Quels sont les numéros, noms et prénoms des professeurs de seconde année ? *3 attributs, 9 tuples*

```
CREATE OR REPLACE VIEW Q1 (numProf, nomProf, prenomProf) AS
SELECT DISTINCT P.numProf, nomProf, prenomProf
FROM Professeur P
    INNER JOIN Enseigne E
        ON P.numProf = E.numProf
    INNER JOIN Etudiant Et
        ON E.numEt = Et.numEt
WHERE anneeEt = 2;
SELECT *
FROM Q1;
```

Créez des vues supprimant les calculs complexes des questions correspondantes du TD5 : Interrogations avancées en SQL.

Q2 À partir du nombre d'étudiants de chaque groupe d'étudiants de seconde année, donnez le nombre moyen d'étudiants. *1 attribut, 1 tuple (9)*

```
CREATE OR REPLACE VIEW Q2 (groupeEt, nbEtudiants) AS
```

```
SELECT groupeEt, COUNT(*)
FROM Etudiant
WHERE anneeEt = 2
GROUP BY groupeEt;
SELECT AVG(nbEtudiants)
FROM Q2;
```

Q3 À partir du nombre d'étudiants à qui chaque professeur enseigne, quel est le plus grand nombre d'étudiants? *1 attribut, 1 tuple (45)*

```
CREATE OR REPLACE VIEW Q3 (numProf, nbEtudiants) AS
SELECT numProf, COUNT(DISTINCT numEt)
FROM Enseigne
GROUP BY numProf;
SELECT MAX(nbEtudiants)
FROM Q3;
```

Q4 À partir de la somme d'heures de cours pour chaque discipline, donnez les codes et libellés des modules ainsi que le pourcentage de ces sommes d'heures par rapport au total des heures de cours dans la discipline correspondante. *3 attributs, 9 tuples*

```
CREATE OR REPLACE VIEW Q4 (discipline, total) AS
SELECT discipline, SUM(heureCMPrev)
FROM Module
GROUP BY discipline;
SELECT code, libelle, heureCMPrev / total AS pourcentagediscipline
FROM Module M
    INNER JOIN Q4
    ON M.discipline = Q4.discipline
WHERE heureCMPrev IS NOT NULL;
```

Q5 À partir, du nombre de professeurs qui enseignent chaque module d'une part, et de la moyenne de test maximale de ces modules d'autre part, donnez les codes des modules, le nombre de professeurs qui les enseignent et la moyenne de test maximale de ces Modules. *3 attributs, 6 tuples*

```
CREATE OR REPLACE VIEW Q51 (code, nbProfs) AS
SELECT code, COUNT(DISTINCT numProf)
FROM Enseigne
GROUP BY code;
CREATE OR REPLACE VIEW Q52 (code, moyMax) AS
SELECT code, MAX(moyTest)
FROM Note
GROUP BY code;
SELECT Q51.code, nbProfs, moyMax
FROM Q51
    INNER JOIN Q52
    ON Q51.code = Q52.code;
```

Q6 À partir de la moins bonne et de la meilleure moyenne de test du module ACSI, donnez les numéros des étudiants et les écarts respectifs entre les éventuelles moyennes de test des étudiants et ces moyennes extrêmes. *3 attributs, 29 tuples*

```
CREATE OR REPLACE VIEW Q6 (moyMin, moyMax) AS
SELECT MIN(moyTest), MAX(moyTest)
FROM Note
WHERE code = 'ACSI';
SELECT numEt, moyTest - moyMin, moyTest - moyMax
FROM Note, Q6
WHERE code = 'ACSI';
```

Q7 Créer une vue de mise à jour permettant d'assurer que les discipline insérées ou modifiées ne peuvent être que Gestion, Informatique ou Maths.

```
CREATE OR REPLACE VIEW Q7 AS
SELECT *
FROM Module
WHERE discipline IN ('INFORMATIQUE', 'GESTION', 'MATHS')
WITH CHECK OPTION;
INSERT INTO Q7 (code, libelle, heureCMPprev, heureCMReal, heureTPPrev, heureTPReal, disciplin
```

Insertion invalide.

```
INSERT INTO Q7 (code, libelle, heureCMPprev, heureCMReal, heureTPPrev, heureTPReal, disciplin
```

Remarques
Cette insertion provoque une erreur.

Q8 Créer une vue de mise à jour permettant d'assurer que les responsables d'un Module inséré ou modifié doivent enseigner ce Module.

V1.

```
CREATE OR REPLACE VIEW Q8 AS
SELECT *
FROM Module M
WHERE RESP IN (
    SELECT numProf
    FROM Enseigne E
    WHERE M.code = E.code
)
WITH CHECK OPTION;
```

V2.

```
CREATE OR REPLACE VIEW Q8 AS
SELECT *
FROM Module M
WHERE (code, RESP) IN (
    SELECT code, numProf
    FROM Enseigne
)
WITH CHECK OPTION;
```

```
ALTER TABLE Enseigne DROP CONSTRAINT fk_Enseigt_code;
ALTER TABLE Enseigne ADD CONSTRAINT fk_Enseigt_code FOREIGN KEY (code) REFERENCES Module(code)
BEGIN
    INSERT INTO Enseigne (code, numProf, numEt) VALUES ('FDD', 3, 2317);
    INSERT INTO Q8 (code, libelle, heureCMPprev, heureCMReal, heureTPPrev, heureTPReal, disciplin
END;
/
```

Remarques

Il est nécessaire de faire l'insertion de l'enseignement avant l'intertion du module avec le responsable devant enseigner ce Module, et cela n'est possible qu'en faisant une transaction et en modifiant la contrainte de clef étrangère afin qu'elle ne soit vérifiée qu'à la fin de la transaction.

Insertion invalide.

```
INSERT INTO Q8 (code, libelle, heureCMPprev, heureCMReal, heureTPPrev, heureTPReal, disciplin
```

Remarques

Cette insertion provoque une erreur.

4.2 Requêtes complexes

Q9 Quels sont les numéros, noms et prénoms des étudiants ayant eu une note dans le module Principes des BD ou dans un de ses sous-modules direct ou pas? *3 attributs, 19 tuples*

```
SELECT DISTINCT Et.numEt, nomEt, prenomEt
FROM Etudiant Et
    INNER JOIN Note N
        ON
            Et.numEt = N.numEt
            AND code IN (
                SELECT code
                FROM Module
                START WITH libelle = 'PRINCIPES DES BD'
                CONNECT BY codePere = PRIOR code
            );
```

Version CTE récursive.

```
WITH RECURSIVE
    DescModule (code) AS (
        SELECT code
        FROM Module
        WHERE libelle = 'PRINCIPES DES BD'
        UNION ALL
        SELECT M.code
        FROM Module M
            INNER JOIN DescModule D ON M.codePere = D.code
```



```

    )
SELECT DISTINCT Et.numEt, nomEt, prenomEt
FROM Etudiant Et
    INNER JOIN Note N
        ON
            Et.numEt = N.numEt
            AND code IN (SELECT code FROM DescModule);

```

Q10 Donnez les libellés des modules et les libellés des modules d'où ils sont directement issus.
2 attributs, 32 tuples

```

SELECT M.libelle, MPere.libelle AS libellePere
FROM Module M, Module MPere
WHERE MPere.code = M.codePere;

```

Q11 Quels sont les groupes de seconde année, dans lesquels un étudiant a obtenu, pour le module Conception de SI, une meilleure note de test que le meilleur étudiant du groupe d'étudiants 3 dans le même Module? *1 attribut, 1 tuple (4)*

V1.

```

WITH T (anneeEt, groupeEt, moyTest, libelle) AS (
    SELECT anneeEt, groupeEt, moyTest, libelle
    FROM Etudiant Et
        INNER JOIN Note N
            ON Et.numEt = N.numEt
        INNER JOIN Module M
            ON N.code = M.code
    WHERE libelle = 'CONCEPTION DE SI'
)
SELECT groupeEt
FROM Etudiant
WHERE
    anneeEt = 2
    AND groupeEt IN (
        SELECT groupeEt
        FROM T
        WHERE moyTest > (
            SELECT MAX(moyTest)
            FROM T
            WHERE
                anneeEt = 2
                AND groupeEt = 3
        )
    )
GROUP BY groupeEt;

```

V2.

```

WITH T (anneeEt, groupeEt, moyTest, libelle) AS (
    SELECT anneeEt, groupeEt, moyTest, libelle
    FROM Etudiant Et

```

```

        INNER JOIN Note N
            ON Et.numEt = N.numEt
        INNER JOIN Module M
            ON N.code = M.code
    WHERE libelle = 'CONCEPTION DE SI'
)
SELECT groupeEt
FROM Etudiant
WHERE
    anneeEt = 2
    AND groupeEt IN (
        SELECT groupeEt
        FROM T
        GROUP BY groupeEt
        HAVING MAX(moyTest) > (
            SELECT MAX(moyTest)
            FROM T
            WHERE
                anneeEt = 2
                AND groupeEt = 3
        )
    )
GROUP BY groupeEt;

```

4.3 Consultation des tables systèmes

En lieu et place du schéma relationnel de la base de données exemple, focalisons-nous plutôt à présent sur les vues système suivantes issues du dictionnaire de données Oracle dont un extrait est présenté ci-après :

User_Tables (table_Name, tablespace_Name, ...)

User_Tab_Columns (table_Name, column_Name, data_Type, ..., data_Length, ...)

User_Cons_Columns (... , constraint_Name, table_Name, column_Name, ...)

User_Constraints (... , constraint_Name, constraint_Type, table_Name, ...)

User_Objects (object_Name, ..., object_Type, ...)

Remarques

Les valeurs de l'attribut `data_Type` correspondent aux types de données Oracle : VARCHAR2, NUMBER, DATE, etc.

La valeur de l'attribut `constraint_Type` peut être P, pour PRIMARY KEY, R, pour REFERENCES, C pour CHECK, etc.

La valeur de l'attribut `object_Type` peut être TABLE, VIEW, FUNCTION, PROCEDURE, TRIGGER, INDEX, etc.

Effectuez les requêtes sur les relations actuelles, pas celles éventuellement dans la corbeille. N'exécutez qu'une seule requête à la fois.

En cas de doute ou de curiosité, n'oubliez pas d'utiliser la commande DESCRIBE.

Q12 Quels sont les noms des attributs de la relation Professeur ? *1 attribut, 7 tuples*

```
SELECT column_Name
FROM User_Tab_Columns
WHERE table_Name = 'PROFESSEUR';
```

Q13 Quelle sont les tables ? *1 attribut, 5 tuples*

V1.

```
SELECT table_Name
FROM User_Tables;
```

V2.

```
SELECT object_Name
FROM User_Objects
WHERE object_Type = 'TABLE';
```

Q14 Quelles sont les noms des contraintes d'intégrité ? *1 attribut, 20 tuples*

```
SELECT constraint_Name
FROM User_Constraints;
```

Q15 Quelles sont les noms des contraintes d'intégrité définies sur des relations ayant au moins un attribut de type VARCHAR2 de 20 caractères ou plus ? *1 attribut, 13 tuples*

```
SELECT constraint_Name
FROM User_Constraints
WHERE table_Name IN (
    SELECT table_Name
    FROM User_Tab_Columns
    WHERE
        data_Type = 'VARCHAR2'
        AND data_Length >= 20
);
```

Q16 Quelles sont, pour les attributs de type NUMBER, les noms des contraintes et les noms des attributs sur lesquels elles portent ? *2 attributs, 15 tuples*

```
SELECT DISTINCT UCC.constraint_Name, UCC.column_Name
FROM User_Cons_Columns UCC
    INNER JOIN User_Tab_Columns UTC
        ON
            UCC.column_Name = UTC.column_Name
            AND UCC.table_Name = UTC.table_Name
WHERE data_Type = 'NUMBER';
```

Q17 Quels sont les noms des contraintes, les noms attributs sur lesquels elles portent et le type de ces attributs, pour les relations ayant au moins un attribut de type NUMBER ? *3 attributs, 26 tuples*

```
SELECT UCC.constraint_Name, UCC.column_Name, data_Type
FROM User_Cons_Columns UCC
     INNER JOIN User_Tab_Columns UTC
           ON
               UCC.column_Name = UTC.column_Name
               AND UCC.table_Name = UTC.table_Name
WHERE UCC.table_Name IN (
    SELECT table_Name
    FROM User_Tab_Columns
    WHERE data_Type = 'NUMBER'
);
```

Q18 Quelles sont les noms des contraintes de clefs primaires ? *1 attribut, 5 tuples*

```
SELECT constraint_Name
FROM User_Constraints
WHERE constraint_Type = 'P';
```

Q19 Quels sont les noms des attributs, et leurs relations correspondantes, d'attributs portant le même nom dans des relations différentes ? *3 attributs, 7 tuples*

```
SELECT UTC1.column_Name, UTC1.table_Name AS Relation1, UTC2.table_Name AS Relation2
FROM User_Tab_Columns UTC1
     INNER JOIN User_Tab_Columns UTC2
           ON
               UTC1.column_Name = UTC2.column_Name
               AND UTC1.table_Name < UTC2.table_Name;
```